

Contents

The Spectral Predictor: Task-Model Alignment Theory and Why It Failed to Predict Grokking	1
1. Prerequisites Recap	1
2. Kernel Regression Basics	2
3. Eigen-decomposition of the Kernel	4
4. Task-Model Alignment	5
5. From Kernel Regression Theory to a Grokking Predictor	7
6. The Experimental Result and Its Interpretation	8
7. Summary — Tying the Chain Together	11

The Spectral Predictor: Task-Model Alignment Theory and Why It Failed to Predict Grokking

A self-contained technical report for the Unified Benchmark of Grokking Predictors thesis

1. Prerequisites Recap

Let us first understand a few basic terms. We will need these terms in every later section. So let us be clear about them now.

1.1 What is grokking

Suppose we train a neural network on some task. The network has a training set and a test set.

First, the training accuracy becomes almost 1. This happens early in training.

But the test accuracy does not improve at the same time. It stays close to chance level for a long time.

Then, after many more epochs, something sudden happens. The test accuracy jumps from near-chance to near-perfect. This jump happens over a short window of epochs, not slowly over the whole training run.

This whole pattern is called **grokking**. Remember: training accuracy becomes high first. Test accuracy becomes high much later, and suddenly.

The epoch at which the test accuracy jumps up is called the **grok epoch**. In this project, we fix a threshold, such as 0.90 test accuracy. The grok epoch is the first epoch where test accuracy crosses this threshold and stays above it.

1.2 What a grokking predictor is supposed to do

Now let us understand what a “predictor” means in this project.

A **grokking predictor** is some quantity. We compute this quantity from the model’s weights, or from the model’s internal representations, at a given checkpoint.

The claim is this: this quantity should change in some clear way *before* the grok epoch. If this claim is true, the predictor is useful. We would know that grokking is coming, only by looking at the model’s internal state. We would not have to wait and watch the test accuracy itself.

For a predictor to be called useful, it must pass two tests. This project checks every predictor against these same two tests.

Test 1 (leads grok). The predictor’s own signal epoch must come *before* the grok epoch. Not at the same epoch. Not after it.

Test 2 (same across seeds). This lead must happen for many different random seeds, not just one seed. If a predictor works for one seed and fails for another, it is not reliable.

This report is about the third predictor in this project’s list of nine: L2 Norm, Dropout, **Spectral**, AGE, HTSR Alpha, Correlation Traps, Weight-PCA, Higher-MI, Commutator Defect. L2 Norm and Dropout were tested first. Both failed Test 1. Now let us study Spectral.

2. Kernel Regression Basics

The Spectral predictor is not something invented only for this project. It comes directly from an existing theorem in kernel regression. So before we write down the predictor’s formula, we must first understand what a kernel is.

2.1 What a kernel is

Suppose we have N input examples. Suppose we have some way to turn every example i into a vector $\phi_i \in \mathbb{R}^d$. This process is called a **feature map**. The vector ϕ_i is called the **representation** of example i .

A **kernel** is a function. It tells us how similar two examples are. We write it as $k(x_i, x_j)$, and we define it as the inner product of the two feature vectors:

$$k(x_i, x_j) = \phi_i \cdot \phi_j$$

In simple words: two examples are called “similar” by this kernel when their feature vectors point in a similar direction, and are of similar size. This idea comes from kernel regression theory. Kernel regression is an older, well-studied area of machine learning. It existed before deep learning, and it still explains a lot about deep learning today.

2.2 Why a trained network’s hidden layer defines a kernel

Now consider our decoder transformer. It is trained on the task $(a + b) \bmod 97$.

Take one frozen checkpoint of this network. Feed every training example through the network. At a fixed position in the sequence, read off the hidden vector. In this project, we read the hidden vector that feeds directly into `output_head`, at the “=” token position. Call this vector $h_i \in \mathbb{R}^{d_{\text{model}}}$, for example i .

This hidden vector is exactly a feature map, in the sense of Section 2.1. So the frozen network, at this one checkpoint, is acting as a feature extractor. We can treat it as defining a kernel over the training examples.

This is the key connecting idea of this whole report. Freeze the network. Its last hidden layer becomes a feature map. That feature map defines a kernel. This is what lets kernel regression theory say something about a trained neural network at all.

2.3 The representation matrix and the Gram matrix

Now let us stack all these hidden vectors together. We get a matrix:

$$\Phi \in \mathbb{R}^{N \times d_{\text{model}}}, \quad \Phi_{i,:} = h_i$$

We call Φ the **representation matrix**. For our task, with $p = 113$, the training set has $N = \lfloor 0.3 \cdot p^2 \rfloor = 3830$ examples. This is because 30% of all p^2 pairs (a, b) are used for training.

Before we do anything more with Φ , we must **centre** it. Centring means: subtract the average row from every row.

$$\Phi_c = \Phi - \bar{\Phi}, \quad \bar{\Phi} = \frac{1}{N} \sum_{i=1}^N \Phi_{i,:}$$

Why do we centre? Centring removes the constant part shared by all representations. After centring, only the part that differs from example to example remains. This is the same idea used when we centre data before computing a covariance matrix, or before doing Principal Component Analysis.

Now we define the **kernel matrix**, also called the **Gram matrix**:

$$K = \Phi_c \Phi_c^\top \in \mathbb{R}^{N \times N}$$

Each entry K_{ij} is the kernel similarity between example i and example j , after centring. This matrix K is a complete picture. It tells us how the frozen network “sees” the relationship between every pair of training examples, at this one checkpoint.

3. Eigen-decomposition of the Kernel

The matrix K has $N \times N$ numbers. That is too many numbers to look at directly, for $N = 3830$. So the next step is to break K down into a small number of directions we can actually understand. This is where eigenvalues and eigenvectors come in.

3.1 Eigenvalues and eigenvectors of K

K is a symmetric matrix, meaning $K = K^\top$. This is true because K is built as $\Phi_c \Phi_c^\top$. Because of this, K has N real eigenvalues, and N eigenvectors that are perpendicular to each other.

We write the eigenvalues as $\eta_1, \eta_2, \dots, \eta_N$, and all of them are ≥ 0 . We write the eigenvectors as $u_1, u_2, \dots, u_N$. Each eigenvector has length 1, and any two different eigenvectors are perpendicular ($u_k \cdot u_l = 0$ for $k \neq l$). Together, they satisfy this equation:

$$K u_k = \eta_k u_k, \quad k = 1, \dots, N$$

What does this equation mean? u_k is a special direction. When we apply K to u_k , the direction of u_k does not change at all. Only its length changes, by the factor η_k . This is why η_k is called an **eigenvalue**, and u_k is called its **eigenvector**.

By convention, we always **order the eigenvalues from largest to smallest**:

$$\eta_1 \geq \eta_2 \geq \dots \geq \eta_N \geq 0$$

This ordering is just a way of labelling things, but it is a very useful one. Now we can talk about “the top k eigenmodes”. This means the k eigenvectors with the largest eigenvalues. The pair (η_k, u_k) together is called the **k -th eigenmode**.

3.2 What “ordering from largest to smallest” means

Think of the eigenvectors $u_1, u_2, \dots, u_N$ as a new set of coordinate axes. These axes are built specially for this one kernel. In this new coordinate system, the kernel’s action becomes very simple. It just stretches the k -th axis by the factor η_k . It does nothing else.

So the eigenvalue η_k tells us one thing clearly: how much of the total structure in K lies along the direction u_k . A large η_1 means one direction dominates. Most of the pattern in the data lies along that one direction, as seen by this kernel. A small η_N means the opposite. That direction barely matters to the kernel.

3.3 Spectral bias — why large-eigenvalue directions are learned more strongly

Now we come to a fact from kernel regression theory. This fact is central to the whole report. It is also where the name “Spectral” predictor comes from.

When a kernel machine is trained on some target function, it does not learn every direction equally fast. This is also true, in the infinite-width limit, for a neural network trained with gradient descent.

The network learns the directions with **large** eigenvalues first, and it fits them most accurately. The directions with **small** eigenvalues are learned more slowly. These directions are harder for the network to represent precisely, because the network’s own geometry gives very little room to them.

This preference is called **spectral bias**. In simple words: the network is naturally biased to represent its own top eigenmodes well. It is naturally biased against representing its low-eigenmode directions well.

Notice something important here. This fact, spectral bias, is about the representation alone. We have not yet said anything about the actual target function we want to learn — that is, we have not yet said anything about the correct answer $(a + b) \bmod p$. Section 4 brings the target function in.

4. Task-Model Alignment

Section 3 told us which directions the kernel represents strongly. Those are the top eigenmodes. Section 3 also told us which directions the kernel represents weakly. Those are the bottom eigenmodes.

Now we ask the next question. Where does our actual target — the correct label $(a + b) \bmod p$ — live among these directions? Is it mostly in the top eigenmodes, or is it spread out everywhere? This question is what Canatar, Bordelon and Pehlevan (2021) call **task-model alignment**.

4.1 Projecting the target onto the eigenvectors

The target for modular addition is a class label, from the set $\{0, 1, \dots, p - 1\}$. We represent each label as a **one-hot vector**. For training example i , with correct label y_i , the one-hot vector $Y_i \in \mathbb{R}^p$ has a 1 at position y_i , and 0 everywhere else.

Stack all these one-hot vectors together. We get the **label matrix**:

$$Y \in \mathbb{R}^{N \times p}$$

Just like we centred Φ in Section 2.3, we now centre Y , column by column:

$$Y_c = Y - \bar{Y}$$

Now we project these centred labels onto the kernel’s eigenvectors, $U = [u_1, \dots, u_N]$:

$$W = U^\top Y_c \in \mathbb{R}^{N \times p}$$

Look at the k -th row of W . Call it $W_k \in \mathbb{R}^p$. This row tells us: how much of the target’s information, for each of the p classes, lies in eigenmode k .

4.2 Task power and its normalized fraction

Now we define the **task power** in eigenmode k . We take the squared length of W_k , added up over all p classes:

$$w_k^2 = \|W_k\|^2 = \sum_{c=1}^p W_{k,c}^2$$

In simple words, w_k^2 answers one question directly: how much of the target’s total “energy” sits in eigenmode k ? A large w_k^2 means eigenmode k carries a lot of information about the label. A small w_k^2 means eigenmode k carries almost no information about the label.

Now we normalize, by dividing by the total power across all eigenmodes. This gives the **fraction of task power** in mode k :

$$p_k = \frac{w_k^2}{\sum_{j=1}^N w_j^2}$$

Two things are true by construction. First, every $p_k \geq 0$. Second, all the p_k values add up to 1: $\sum_{k=1}^N p_k = 1$. So p_k behaves like a probability distribution, spread over the N eigenmodes.

4.3 Cumulative power $C(k)$

We now reach the single most important quantity in this report. This is the exact quantity named in the claim we are studying. We call it the **cumulative power**, and its formula is:

$$C(k) = \sum_{j=1}^k p_j$$

In simple words: $C(k)$ tells us what fraction of the target’s total variance is captured, if we keep only the top k eigenmodes and throw away the rest. “Top k ” means the k eigenmodes with the largest eigenvalues, η_1 through η_k , from Section 3.1.

Since every $p_j \geq 0$ and they add up to 1, $C(k)$ only increases as k increases. It starts at $C(0) = 0$. It ends at $C(N) = 1$.

From $C(k)$, we get two useful signals, and both are actually used in this project’s benchmark:

$$k_{90} = \min\{k : C(k) \geq 0.90\}, \quad k_{95} = \min\{k : C(k) \geq 0.95\}$$

k_{90} answers a direct question: how few of the top eigenmodes do we need, to explain 90% of the target’s variance? A small k_{90} means the target’s information is concentrated in a few top directions. A large k_{90} , close to N , means the target’s information is spread thinly across almost the whole spectrum.

4.4 Why fast-rising $C(k)$ means good generalization — Canatar et al.’s theory

We can now state the theorem behind the Spectral predictor, precisely.

Canatar, Bordelon and Pehlevan (2021, *Nature Communications*, “Spectral bias and task-model alignment explain generalization in kernel regression and infinitely wide neural networks”) worked out an exact formula. This formula gives the generalization error of kernel regression. It is written entirely in terms of the two objects we have already built: the eigenvalues η_k from Section 3, and the task power fractions p_k from Section 4.2.

Their main conclusion, in plain words, is this: generalization from a limited training set is good, and needs fewer examples, exactly when the target’s power sits mostly in the top eigenmodes of the kernel. Why the top eigenmodes specifically? Because those are the same modes that spectral bias (Section 3.3) makes the model learn fastest and most accurately. This match between “where the target’s power sits” and “what the model represents well” is called **task-model alignment**. The task is called “aligned” with the kernel when a small k already gives $C(k)$ close to 1.

Let us connect this back to Section 3.3, step by step. Spectral bias says: the model represents top eigenmodes strongly, and bottom eigenmodes weakly. Now suppose the target’s information also sits mostly in the top eigenmodes — that is, $C(k)$ rises quickly for small k . Then the model’s natural learning order, driven by spectral bias, matches exactly where the useful information is. So the model can fit the target well, from relatively few examples.

Now suppose the opposite. Suppose the target’s power is spread across many eigenmodes, including the low-eigenvalue ones the model finds hard to represent. Then generalization will be poor, or it will need far more training data. This is the exact meaning of the sentence in the original claim: “ $C(k)$ rising toward 1 for small k signals good alignment and fast generalization.”

5. From Kernel Regression Theory to a Grokking Predictor

Sections 2 to 4 covered pure kernel regression theory. We have not said anything about grokking or epochs yet. This section states the exact hypothesis that turns this theory into a testable predictor.

5.1 The precise hypothesis being tested

Take one frozen training checkpoint. We can compute the network’s representation kernel K (Section 2). We can compute its eigenvalues and eigenvectors (Section 3). We can compute the cumulative power curve $C(k)$, and the values k_{90} and k_{95} (Section 4).

Now do this at many checkpoints, across the whole training run. We get a time series — a sequence of values, one set per checkpoint.

The hypothesis we are testing is this: **does $C(k)$ rise, and does k_{90} shrink, before the grok epoch — not at it, not after it, but clearly before it?**

We also track a single number, which summarizes the same idea. We call it the **alignment score**:

$$\text{alignment_score} = \sum_k \tilde{\eta}_k p_k, \quad \tilde{\eta}_k = \frac{\eta_k}{\sum_j \eta_j}$$

This score weights each mode’s task power by its normalized eigenvalue. So the alignment score becomes high exactly when large task power and large eigenvalue happen at the same eigenmode. In terms of this one number, the hypothesis becomes simpler to state: **does the alignment score reach its maximum before the grok epoch?**

5.2 Why this would count as an early-warning signal, if true

Suppose task-model alignment really does build up slowly, and finishes *before* the test-accuracy jump. Then, by watching k_{90} shrink, or by watching the alignment score climb, we would know that grokking is coming. We would know this without waiting to see the jump in test accuracy itself.

This is exactly what Test 1 of Section 1.2 requires. The signal epoch — here, the epoch where k_{90} hits its minimum, or where the alignment score hits its maximum — must come before the grok epoch. If this lead also holds for every seed (Test 2), then Spectral would be a genuinely useful grokking predictor.

This is a real, meaningful hypothesis to test. It is not something made up only for grokking. It comes from an established, general theorem about kernel regression. If the hypothesis were true, it would tell us something important: that grokking on modular addition is, underneath, a kernel-regression-style alignment effect.

6. The Experimental Result and Its Interpretation

Now let us look at what actually happened, when this hypothesis was tested.

6.1 Setup

The Spectral predictor was tested on the decoder transformer trained on $(a + b) \bmod p$, with $p = 113$. This gives $N = \lfloor 0.3 \cdot 113^2 \rfloor = 3830$ training examples.

Training was run for **5 random seeds**. Each seed trained for **40,000 epochs**. All training was done on the Apple Silicon **MPS** backend. During each run, 24 checkpoints were saved. The Spectral metrics were then computed from these saved checkpoints. No retraining was needed for this step, because training and analysis are kept separate in this project’s pipeline.

One implementation detail is worth mentioning here, since it affects correctness, not theory. The function `torch.linalg.eigh`, which computes eigenvalues and eigenvectors, does not run on the MPS backend. So, for every checkpoint, the 3830×3830 Gram matrix was first moved to the CPU, and then converted to float64 precision, before the eigendecomposition was computed. This is only a numerical step. It changes nothing about the theory in Sections 2 to 5.

Grokking was confirmed on all 5 seeds. The grok epoch was different for each seed. Mean grok epoch was 13218.8. Standard deviation was 5900.6. Minimum was 6988. Maximum was 24021.

6.2 The finding: $C(k)$ rises only after grokking, on every seed

Here is the result, for all 5 seeds.

seed	k_{90} : first → last	$k_{90,\min}$ epoch	alignment score: first → last	alignment _{max} epoch	grok epoch
0	3448 → 3369	25232	0.0002 → 0.0085	25232	14474
1	3447 → 3404	10040	0.0002 → 0.0087	15917	6988
2	3448 → 3398	25232	0.0002 → 0.0085	39999	10418
3	3450 → 3346	39999	0.0002 → 0.0085	39999	10193
4	3448 → 3350	39999	0.0002 → 0.0087	39999	24021

Remember, $N = 3830$ here.

Two things stand out from this table. Let us take them one at a time.

First point. k_{90} barely moves. It starts near 3448. This is close to $0.90 \times N$. By the end of training, it only falls to somewhere between 3346 and 3404. That is a drop of only about 2 to 3 percent of N . So throughout training, we need almost 90% of all the eigenmodes to explain 90% of the target’s variance. The target’s power never concentrates into a small set of top eigenmodes. It stays spread almost uniformly across nearly the entire spectrum, from the first checkpoint to the last.

Second point. This is the main finding of the whole report. Look at the epoch where k_{90} reaches its minimum. Look also at the epoch where the alignment score reaches its maximum. In every single one of the 5 seeds, both of these epochs come **at or after** the grok epoch. Never before it.

Let us check the closest case, seed 1. Here, $k_{90,\min}$ happens at epoch 10040. The grok epoch for this seed is 6988. So even this closest case is still *after* grokking, by 3052 epochs. In three of the five seeds — seeds 2, 3, and 4 — both $k_{90,\min}$ and $\text{alignment}_{\max}$ are simply stuck at the very last saved checkpoint, epoch 39999. This means the slow rise in alignment had not even finished, by the end of a 40,000-epoch run.

Now let us apply the two-test check from Section 1.2. **Test 1 fails in all 5 out of 5 seeds.** The alignment signal never once leads the grok epoch. So Test 2 does not even need to be checked — there is no lead at all for Test 2 to be consistent about.

6.3 Why this is a valid negative result, not an implementation bug

It is important to be precise here. Let us be clear about what this finding says, and what it does not say.

The implementation in Sections 2 to 4 follows Canatar et al.’s equations exactly. The kernel is built the same way. The centring is done the same way. The eigen-decomposition is done the same way. The projection of one-hot labels onto eigenvectors is done the same way. The cumulative-power formula is exactly the same. There is no shortcut and no approximation anywhere.

The alignment score does rise steadily across training, in every seed. It rises from about 2×10^{-4} , up to somewhere between 8.5×10^{-3} and 8.7×10^{-3} . This tells us the implementation is correctly picking up a real change in the representation, as training goes on. The predictor is working exactly as it should. It is only that what it detects rises too late to be a useful early-warning signal.

This is how we know it is not a bug, and not a code error. A bug would usually give a flat signal, or a noisy signal with no clear trend, or a signal that fails an obvious sanity check. During development, a separate smoke test was run on an untrained, 100-epoch model, which had not yet grokked. On this model, $k_{90} \approx 0.9N$, and the entropy was close to $\log N$. This is exactly what we expect before any learning has concentrated the representation — the task power should look near-uniform. So the code passes this basic check.

Here, in the real result, we see a real, steady, correctly computed rise in alignment. It is just that this rise lands after the grok epoch, in all 5 seeds, with no exception at all. This exact pattern — correct implementation, real signal, wrong timing — is what we call a **valid negative result**. The predictor is well-built and well-motivated by theory. It was tested honestly. It simply does not work on this task.

6.4 What this implies about the actual mechanism of grokking on modular addition

Canatar et al.’s theory belongs to what is called the **kernel regression regime**. This is also called the lazy regime, or the NTK (Neural Tangent Kernel) regime. In this regime, we treat the network’s representation kernel as if it stays fixed throughout training. Only the final readout weights, sitting on top of this fixed feature map, are assumed to change. In this regime, generalization depends only on how the target’s

power lines up with this one fixed kernel’s eigenspectrum — exactly as we built up in Sections 3 and 4.

Grokking on modular addition works differently. This project’s data, and the wider literature on this task, both point to the same conclusion: grokking here is a **feature-learning phenomenon**, not a fixed-kernel phenomenon.

The network does not keep a fixed representation kernel, and then slowly line up the target with it. Instead, over the course of training, the network’s own hidden representations reorganize themselves. They build a specific, structured circuit — the well-known Fourier-multiplication circuit for modular addition. It is the *building of this circuit* that causes the sudden jump in test accuracy. It is not a slow alignment of a fixed kernel to the task.

We can see this directly in the numbers of Section 6.2. The centred representation Gram matrix K stays high-rank — that is, k_{90} stays close to N — right through the grok epoch. It only drops a little, and only well after grokking has already happened. There is no collapse in the rank of the representation kernel before the test-accuracy jump. There is no concentration of task power into a small set of top eigenmodes, before the jump either.

So the kernel-regression picture of Sections 2 to 5, where a fixed kernel’s spectrum explains generalization, is simply the wrong lens for what is actually happening, mechanically, during grokking on this task. The real phenomenon lives in the feature-learning dynamics that build the Fourier circuit. It does not live in a static kernel’s eigenstructure.

7. Summary — Tying the Chain Together

Let us now put the whole chain together, one small step at a time.

Eigenvalues η_k come from the centred representation Gram matrix, $K = \Phi_c \Phi_c^\top$. They rank the directions in representation space. Largest eigenvalue means the direction is represented most strongly (Section 3.1).

Eigenvectors u_k are these directions themselves. Spectral bias tells us that the network represents the top ones — the ones with large η_k — most strongly (Section 3.3).

Task power $w_k^2 = \|U_k^\top Y_c\|^2$, normalized to p_k , measures how much of the target function’s information sits along each eigenvector direction (Section 4.2).

Cumulative power $C(k) = \sum_{j \leq k} p_j$ measures how much of the target is explained, once we keep only the top k eigenmodes (Section 4.3).

Alignment is Canatar et al.’s (2021) main claim: a fast-rising $C(k)$ — meaning target power concentrated in the top eigenmodes, where the model is strong — should predict good, sample-efficient generalization (Section 4.4).

Predicted timing, under the grokking-predictor hypothesis of Section 5, was: $C(k)$ should rise, and k_{90} should shrink, *before* the grok epoch. This would give us an

early-warning signal.

Actual timing, found in Section 6, was different. k_{90} barely moved at all — it stayed near $0.9N$ throughout training. Its minimum, and the alignment score’s maximum, both landed *at or after* the grok epoch, in all 5 seeds. Never before it.

Falsification, stated simply

Sections 3 and 4 together tell us what “alignment” means. It is a property of the target’s power distribution, measured against a fixed kernel’s eigenspectrum. Alignment is high exactly when the eigenmodes the model represents strongly (Section 3, large η_k) are the same eigenmodes carrying the target’s power (Section 4, large p_k).

The Spectral predictor’s code follows this definition faithfully, in every respect. It correctly detects a real, steadily rising alignment score across training, in every one of the 5 seeds.

So what falsifies its usefulness as a grokking predictor? Only one thing: timing. Not correctness. This rise in $C(k)$, and in the alignment score, lands at or after the grok epoch, on all 5 seeds, with no exception. Because of this, the predictor fails Test 1 of Section 1.2 outright. It cannot act as an early-warning signal. By the time alignment has actually risen, generalization has already happened.

This does not mean the predictor was implemented wrongly. It was implemented correctly. What it means is this: task-model alignment, in the fixed-kernel sense defined in Sections 3 and 4, is not the mechanism driving grokking on this task. The real mechanism is the feature-learning process that builds the Fourier circuit. A kernel-regression-style predictor, by its very construction, cannot see this circuit forming in advance.